COMP3069 Computer Graphics

Lab 10

Originally prepared by Dr Kristian Spoerer of UNUK

Modified by Prof Sean He of UNNC

Learning Outcomes

After Lab 10, you are expected to be able to

1. Recognise the jaggies on polygon edges caused by aliasing.
2. Write simple OpenGL code for querying the supported number of Multisamples.
3. Write simple GLFW code for enabling Multisampling.
4. Demonstrate that you can use the OpenGL implementation of Multisampling.
5. Experiment with Multisampling by adjusting the number of samples per pixel.
6. Observe the improved smoothing when using more samples per pixel and relate the number of samples to the smoothness of the render.
7. Compare the different render quality using different numbers of samples per pixel and explain the increased computational cost.

Setup

In the COMP3069 directory, create a new project called

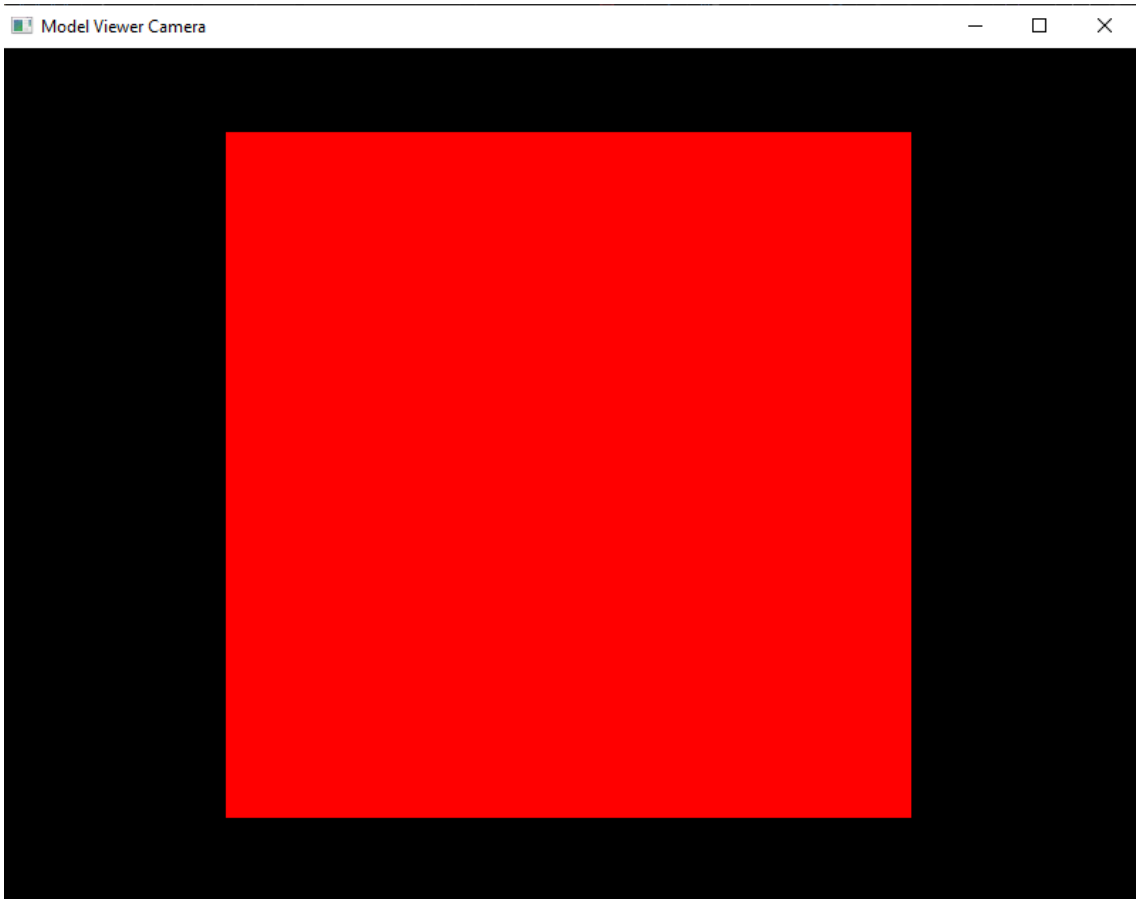
`Lab10`

Download Lab10.cpp from Moodle and copy the code to the Lab10.cpp in your new project.

Download camera.h, ModelViewerCamera.h, shader.h, util.h, and window.h from Moodle. Copy them to your project and add them to the Header Files in the Solution Explorer.

Download mvp.vert and col.frag from Moodle, copy them to your project, and add them to the Source Files in the Solution Explorer.

If you run the program, then the following window should appear



Use the ModelViewerCamera to rotate the camera around the multicoloured cube using the arrow keys, and modify the distance between the camera and cube using F and R keys.

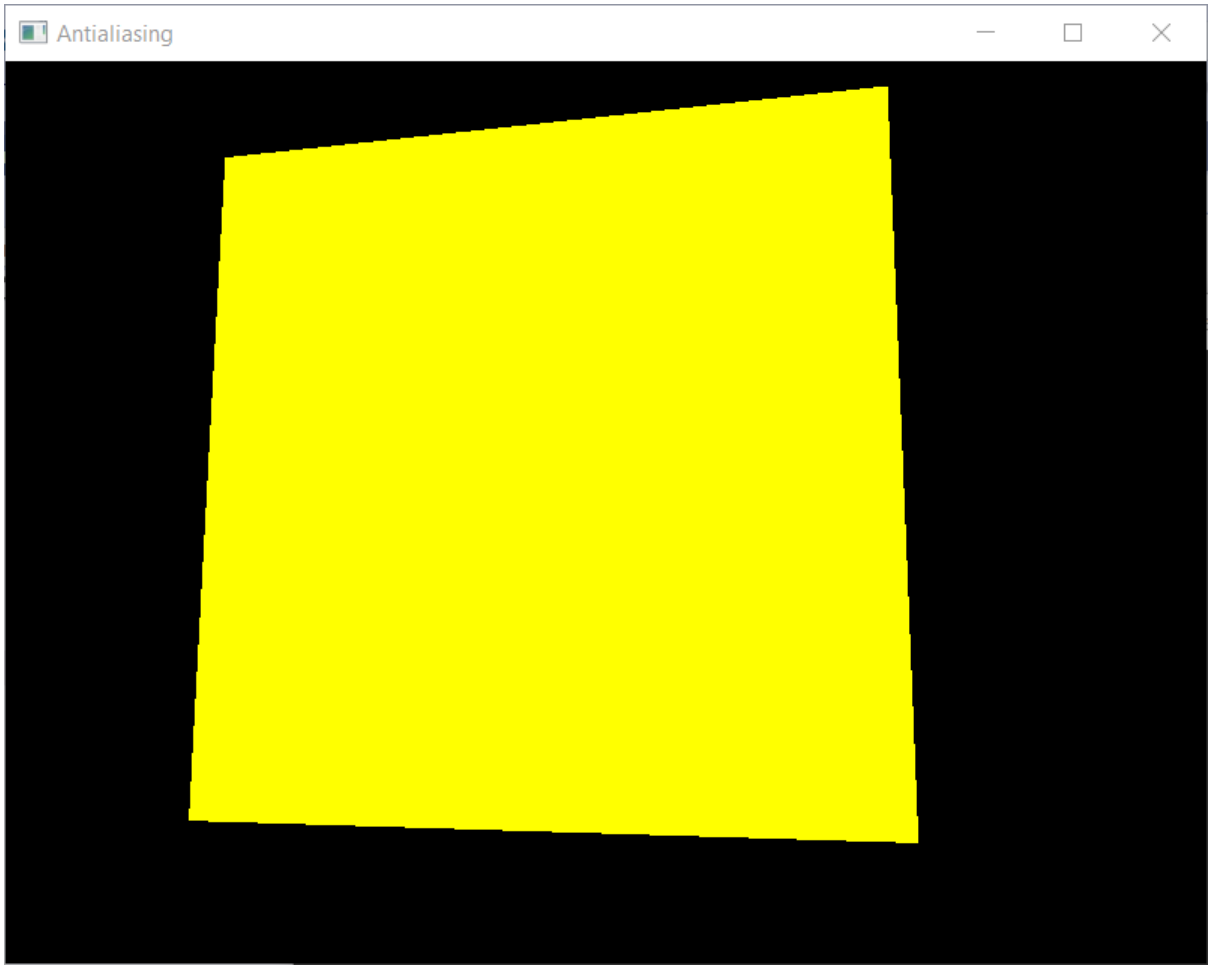
This lab is very short and simple, which will show you how to use the OpenGL implementation of Multisampling antialiasing.

Enable Multisampling

Initialise the camera pitch and yaw so that the camera is pointing in the correct direction.

```
127     unsigned int shaderProgram = LoadShader("mvp.vert", "col.frag");
128
129     InitCamera(Camera);
130     Camera.Pitch = -8.5f;
131     Camera.Yaw = -12.f;
132     MoveAndOrientCamera(Camera, cube_pos, cam_dist, 0.f, 0.f);
133
```

Run the program and you will see the cube from this angle.



The ‘jaggies’ caused by aliasing are clearly visible along all four edges of the yellow face of the cube.

We are going to use the OpenGL function glGetIntegerv()

```
void glGetIntegerv(GLenum pname, GLint * data)
```

here `pname` is the parameter which we want to query, and `data` stores the value of the parameter. We want to use `GL_MAX_SAMPLES` as `pname`, which is the maximum supported number of samples for multisampling and is guaranteed to be at least 4.

In main(), print the number of supported Multisamples using printf. It is important that you have first created a window and also called gladLoadGLLoader() before calling glGetIntegerv(). It will look like this

```
123     GLFWwindow* window = CreateWindow(800, 600, "Antialiasing");
124
125     gladLoadGLLoader((GLADloadproc)glfwGetProcAddress);
126
127     int numMultiSamples;
128     glGetIntegerv(GL_MAX_SAMPLES, &numMultiSamples);
129     printf("numMultiSamples %d\n", numMultiSamples);
130
131     unsigned int shaderProgram = LoadShader("mvp.vert", "col.frag");
```

When you run the program, you should see something like this printed on your console

```
numMultiSamples 32
```

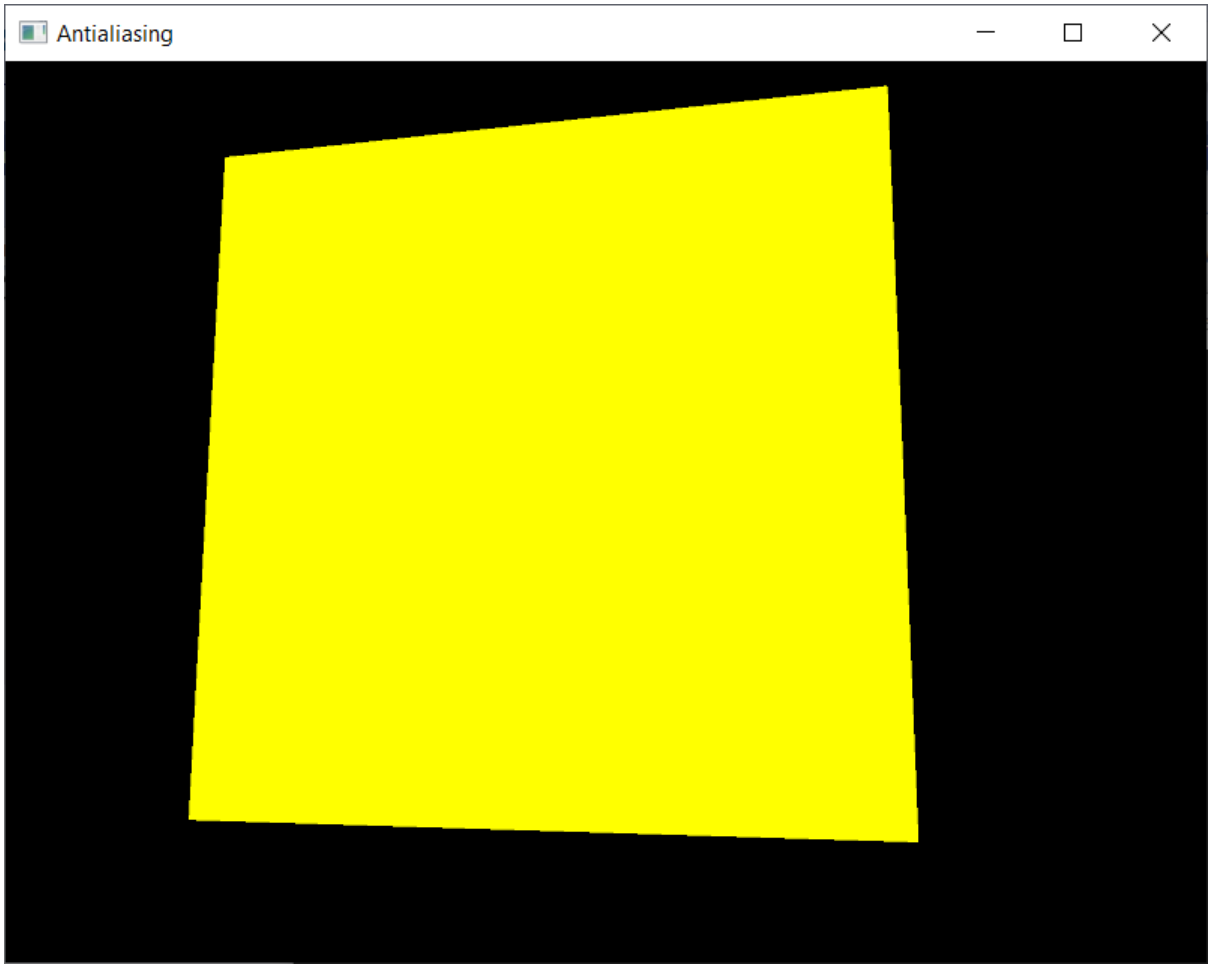
This means that 32x Multisampling is supported. It may be different for your specific computer that you are using now.

Stop the program executing so that we can use this information to enable Multisampling.

In window.h and in the CreateWindow() function, add a single line of code to enable 2x Multisampling. It is important to enable multisampling before creating the window. It should look like this

```
14      glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
15
16      glfwWindowHint(GLFW_SAMPLES, 2);
17
18      GLFWwindow* window = glfwCreateWindow(w, h, title, NULL, NULL);
```

When you run the program you should see this



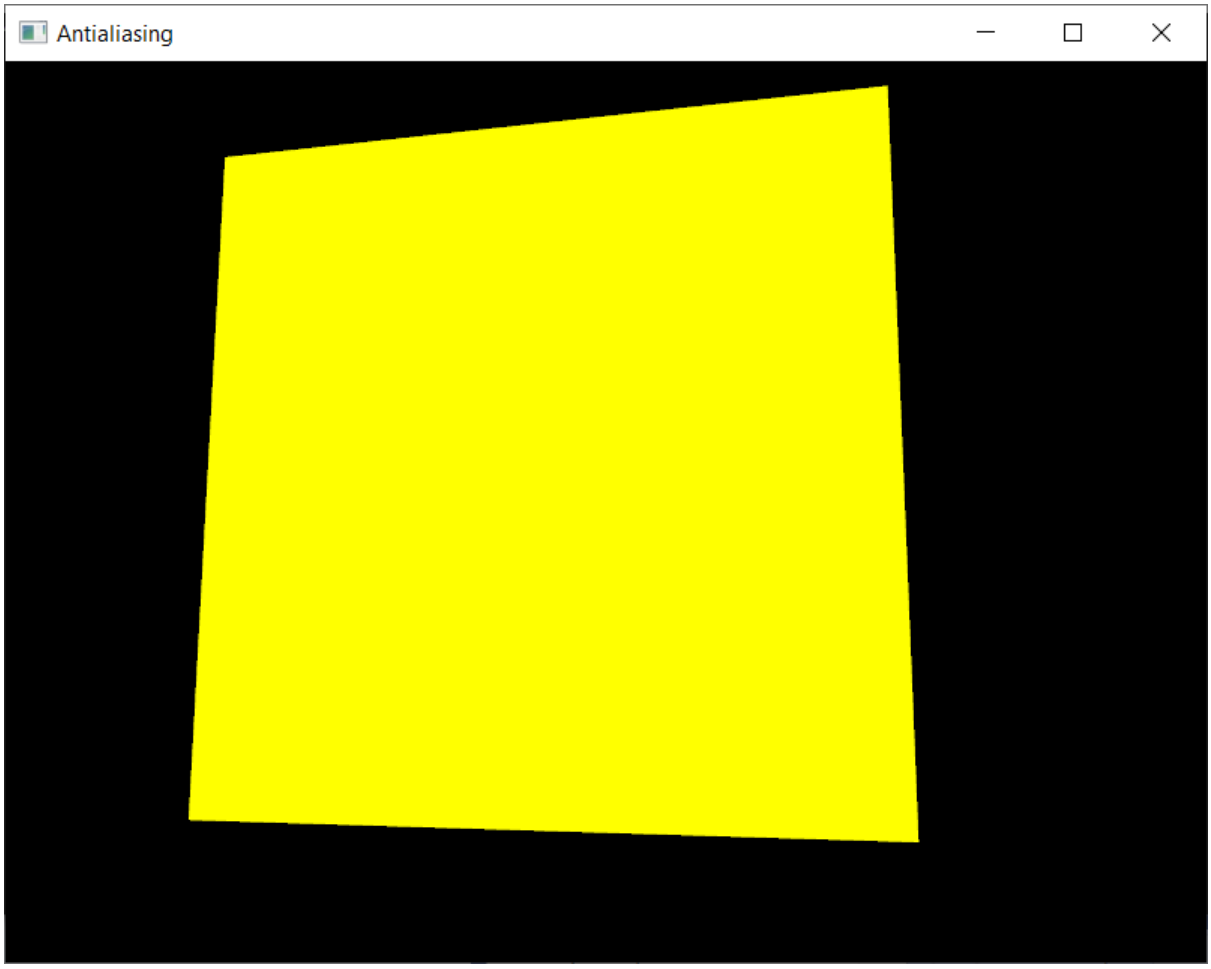
The aliasing has been smoothed a little bit.

In fact, on the bottom edge you can see the original aliasing with the additional sample half way between each jaggie, partially smoothing the render.

Now increase the number of samples to 4x Multisampling

```
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
    glfwWindowHint(GLFW_SAMPLES, 4);
    GLFWwindow* window = glfwCreateWindow(w, h, title, NULL, NULL);
```

Run the program you should see this

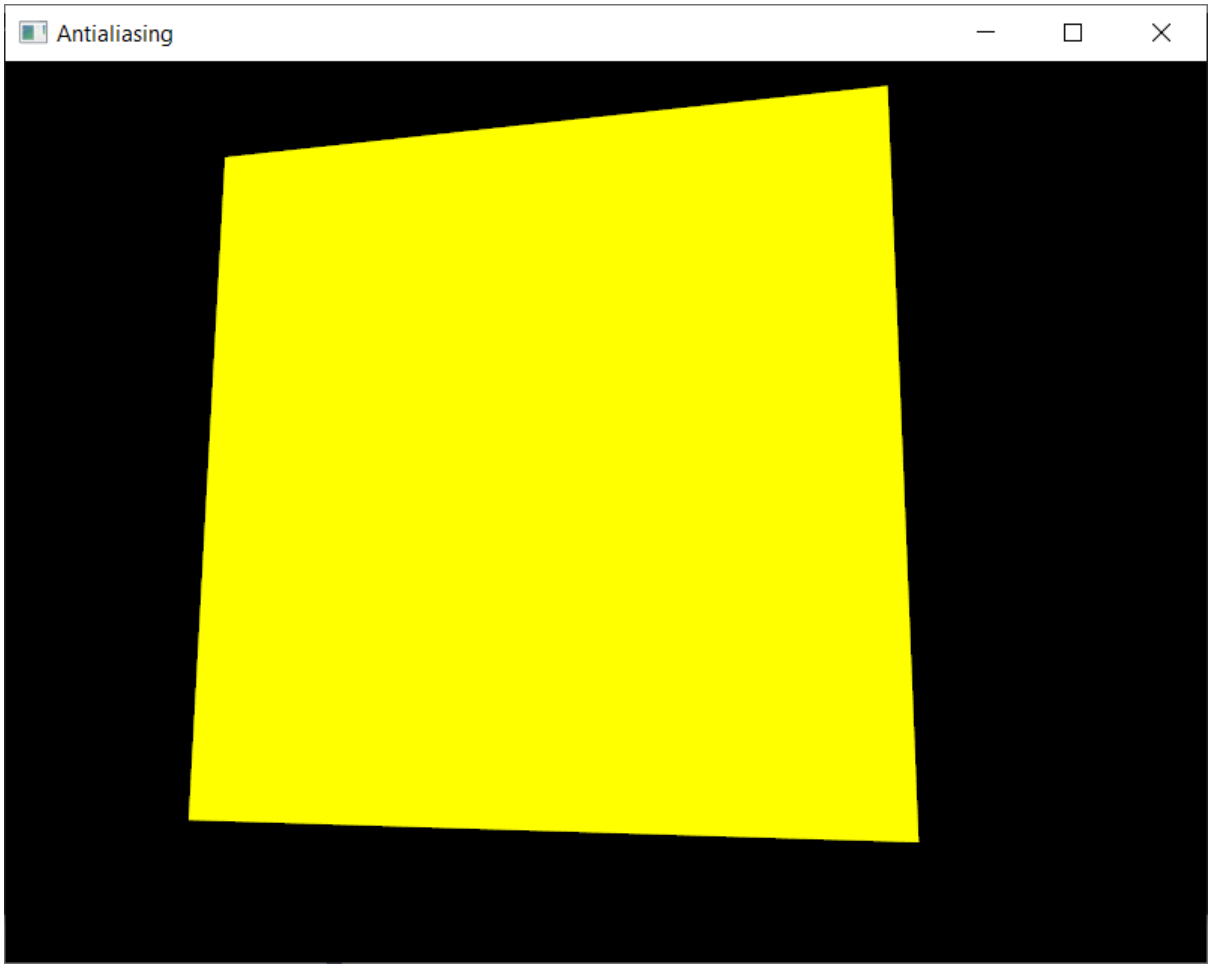


Which is a bit smoother.

Increase the number of samples to 8x Multisampling

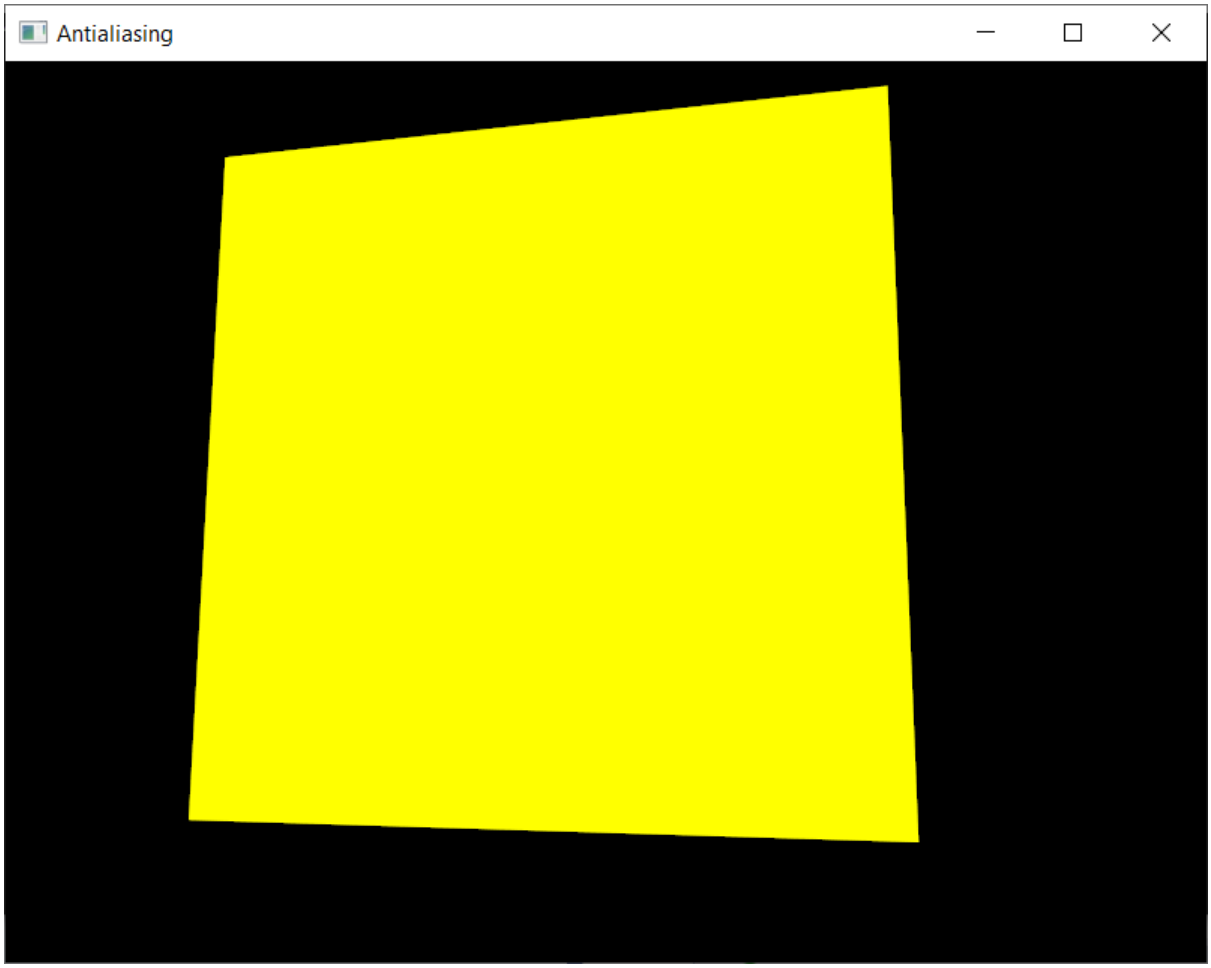
```
15
16      glfwWindowHint(GLFW_SAMPLES, 8);
17
18      GLFWwindow* window = glfwCreateWindow(w, h, title, NULL, NULL);
```

Run the program you should see this

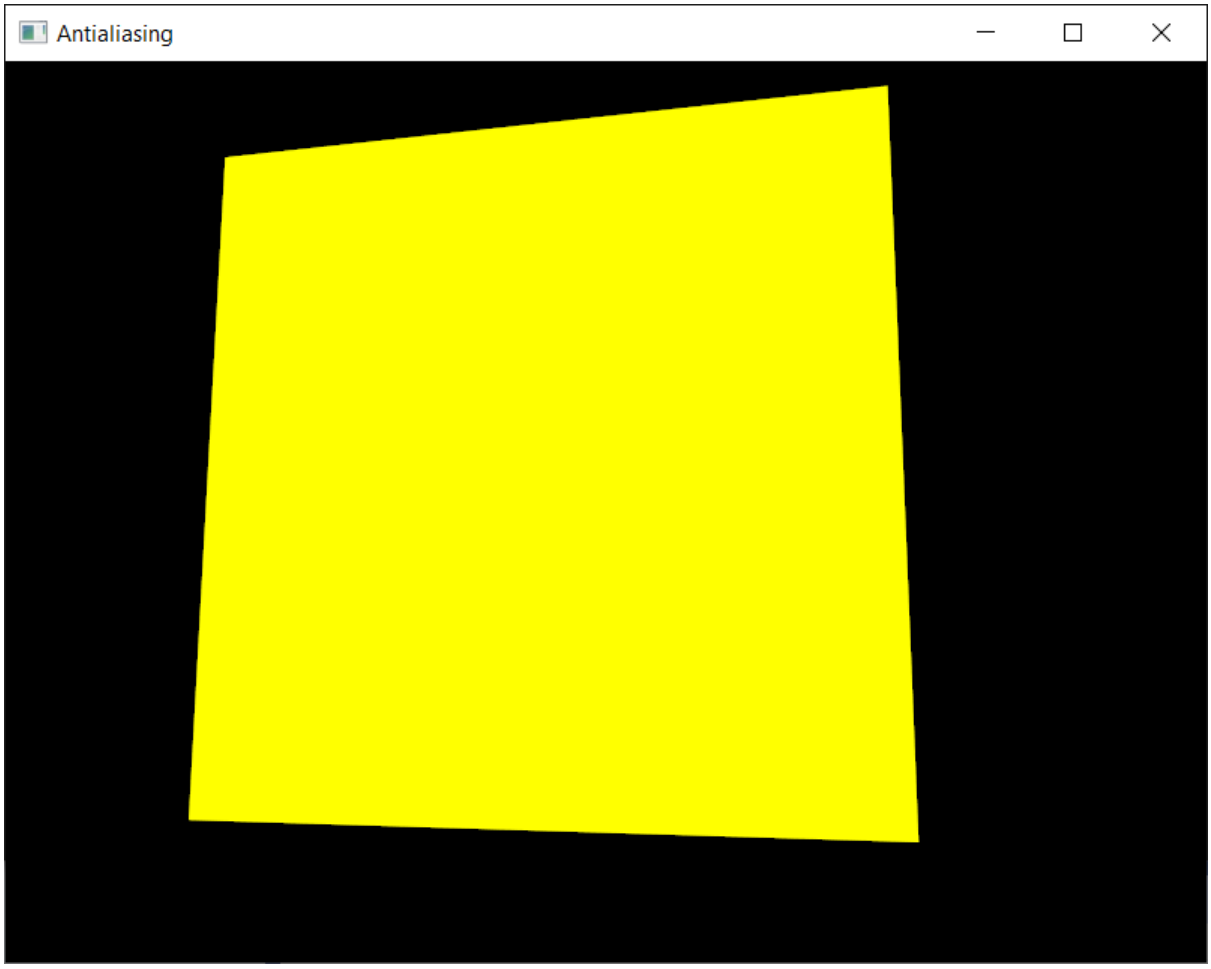


Which is even smoother.

Running with 16x Multisampling produces this



And 32x Multisampling produces this



Try to use the maximum available samples and you should see the resulting render appear smoother.

As you can see, using OpenGL Multisampling is very simple, and the results are excellent.

Demo – show your running program to a member of the teaching team.